

Лабораторная работа №11

Инструментальные средства веб-разработки. Фреймворк Django

Выполнили: студенты группы ИСТ-52

Пальчиков Семён и Львов Роман

Цель работы

Приобретение комплексных навыков разработки веб-приложений на языке Python с использованием фреймворка Django.

- Инициализацию проекта и управление зависимостями через современный ин-струмент uv
- Проектирование архитектуры приложения по шаблону MVT
- Освоение технологий контейнеризации для обеспечения переносимости среды

Развернуть его в изолированном контейнере и

Модель данных (models.ru)

```
from django.db import models
class CalculationHistory (models.Model):
    num1 = models.FloatField()
    num2 = models.FloatField()
    operation = models.CharField(max_length=1)
    result = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.num1} {self.operation} {self.num2} = {self.result}"
```

Представление (views.py)

```
from django.shortcuts import render
from .models import CalculationHistory

def index(request):
    result=None
    error=None

    if request.method == 'POST':
        try:
            num1=float(request.POST.get('num1'))
            num2=float(request.POST.get('num2'))
            operation = request.POST.get('operation')

            if operation == '+':
                result = num1 + num2
            elif operation == '-':
                result = num1 - num2
            elif operation == '*':
                result = num1 * num2
            elif operation == '/':
                if num2 == 0:
                    error = "Деление на ноль невозможно"
                else:
                    result = num1 / num2

            if error is None and result is not None:
                CalculationHistory.objects.create(
                    num1=num1,
                    num2=num2,
                    operation=operation,
                    result=result
                )
        except (ValueError, TypeError):
            error="Пожалуйста, введите корректные числа"

    history = CalculationHistory.objects.all().order_by('-created_at')

    return render(request, 'calculator/index.html',{
        'result':result,
        'error':error,
        'history':history.
```

Создаем интерфейс для калькулятора (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>Калькулятор на Django</title>
  <style>
    body {font-family: sans-serif; display: flex; justify-content: center; padding-top: 50px;}
    form {border: 1px solid #ccc; padding: 20px; border-radius: 10px; background: #f9f9f9;}
    input, select {padding: 5px; margin: 5px 0;}
    button {width: 100%; padding: 10px; background: #28a745; color: white; border: none; cursor: pointer;}
    .result {margin-top: 10px; font-weight: bold; color: blue;}
    .error {color: red;}
    .history-block {margin-top: 30px; width: 300px;}
    .history-item {border-bottom: 1px solid #eee; padding: 5px 0; color: #555; font-size: 0.9em;}
  </style>
</head>
<body>
  <form method="post">
    {% csrf_token %}
    <input type="number" name="num1" step="any" required placeholder="Число 1">
    <br>
    <select name="operation">
      <option value="+">+</option>
      <option value="-">-</option>
      <option value="*">*</option>
      <option value="/">/</option>
    </select>
    <br>
    <input type="number" name="num2" step="any" required placeholder="Число 2">
    <br>
    <button type="submit">Посчитать</button>

    {% if result is not None %}
    <div class="result">Результат: {{ result }}</div>
    {% endif %}
    {% if error %}
    <div class="error">{{error }}</div>
    {% endif %}
  </form>

  <div class="history-block">
    <h4>История (последние 5):</h4>
```

Импортируем из django.urls функцию path

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index')
]
```

В файле config/urls.py подключаем пути

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('calculator.urls')),
]
```

Применить миграции для создания базы данных

В терминале вводим команду

```
uv run python manage.py makemigrations
```

Затем вводим (uv run python
manage.py migrate)

```
uv run python manage.py migrate
```


Для контейнеризации создаем Dockerfile. В нем пишем инструкцию

```
FROM python:3.14-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY . .
ENV PATH="/app/.venv/bin:$PATH"
EXPOSE 8000
CMD ["sh", "-c", "python manage.py makemigrations && python manage.py migrate && python manage.py runserver 0.0.0.0:8000"]
```

**Создаем файл .dockerignore,
чтобы не копировать мусор
внутри образа.**

В него вписываем следующие строки

```
.git  
.venv  
__pycache__  
*.pyc  
db.sqlite3
```

Для сборки образа вводим в терминале команду

`docker build -t my-calculator .`

После успешной сборки запускаем контейнер командой

```
docker run -p 8000:8000 my-calculator
```

Чтобы решить проблему сохранения между перезапусками, создаем файл `docker-compose.yml`

В нем пишем следующие строки

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app/data
      - ./db.sqlite3:/app/db.sqlite3
```

Запускаем через кнопку “Run (All) Service” или через команду `docker compose up`